THESIS TITLE

A PREPRINT

Knud Rasmus Sønderbek

April 9, 2026

ABSTRACT

Some super cool abstract of all my findings

1 Introduction

Neural networks continue to achieve remarkable performance across domains ranging from medical diagnosis to autonomous systems. This increase in performance has, however, come with a corresponding increase in complexity, making these systems less interpretable to those who build and deploy them. As neural networks enter high-stakes environments, where errors carry legal, financial, or human cost, interpretability becomes a necessity. The concern is not merely technical but ethical: a fundamental principle of accountability holds that the creator of a tool can only be held responsible for its outcomes to the extent that those outcomes could have been reasonably foreseen. A system whose internal reasoning cannot be understood is, by definition, one whose failures cannot be anticipated.

Mechanistic interpretability aims to reverse-engineer the internal computations of neural networks into their simplest forms, such that they can be expressed as humanly interpretable algorithms.
Many of the current popular methods within this space, such as sparse autoencoders [Cun+23], circuit analysis, and probing, primarily operate in activation space. However, each of these methods comes with its own set of limitations. Sparse autoencoders treat features as independent directions and discard the geometric structure between them, despite growing evidence that this structure carries meaningful information (Leask et al., 2025; Mendel, 2024; Engels et al., 2024). Moreover, sparse autoencoders are trained to reconstruct activations and do not to explicitly identify the computational units that produced them. [Lea+25; Cha+25] The resulting decomposition may be functionally equivalent at a given layer, while not resembling the mechanisms that the network employs. Circuit-level methods face the deeper issue that neural networks are densely connected, meaning any given output can be equally well attributed to a multitude of causal pathways, leaving the choice of circuit underdetermined.

A more recent line of work shifts the object of decomposition from activations to parameters. More concretely, these methods aim to decompose a network's weights into a sum of simpler components, each of which corresponds to a concrete "mechanism". This reframing sidesteps several of the issues above: mechanisms are no longer constrained to specific architectural components, such as neurons or attention heads, and can instead be distributed across multiple architectural components. Moreover, they decompose the network into functional units rather than representational directions, which may further enhance their interpretability.
Recently proposed methods guided by the principle of minimum description length have shown promising results in this direction, aiming to decompose a network into parameter components that are faithful to the original weights, minimal in the number required per input, and individually as simple as possible.
One such method, Stochastic Parameter Decomposition (Bushnaq et al., 2025), decomposes networks into rank-one subcomponents. Since not all components are relevant for every input, the method learns a function that estimates each component's causal importance for a given input. However, each component's importance is calculated independently, relying only on its own activation. As noted in the original work, this is likely too restrictive for non-toy models, as it prevents each component's importance from being informed by the state of other components. This work replaces the independent importance functions with a graph neural network that determines each component's importance conditioned on its relationships to all other components, while preserving explainability through its generalised additive structure. Additionally the use of an additive importance function, generalizes the decomposition to k-dimensional components. This generalizsation is motivated by the growing evidence, that transformer models, learn features which occupy irreducible multidimensional subspaces.

[Bra+25]

2 Related work

This is the related work of the thesis.

3 Method

3.1 Background on APD

The initial inspiration for shifting the focus from activations to parameters came from Attribution-based Parameter Decomposition (APD) [Bra+25]. APD decomposes the weights of a network into a set of parameter components: vectors in parameter space that satisfy three desirable properties.

Let $f(x, W)$ be a neural network with weight matrices $W = \{W^1, \ldots, W^L\}$. A set of parameter components should satisfy the following properties:

- **Faithfulness**: The parameter components should sum to the parameters of the original network.
- **Minimality**: As few parameter components as possible should be required to replicate the network's output on any given input.
- **Simplicity**: Each component should span as few weight matrices and as few ranks as possible.

If a set of parameter components satisfies these properties, they are said to comprise the network's mechanisms [Bra+25].

However, APD suffers from several practical limitations. Each parameter component is a full vector in parameter space $\mathbb{R}^{|\theta|}$, where $|\theta|$ is the total number of parameters in the network, thus increasing the memory cost to $O(\theta \cdot C)$, where C is the number of components, making the method computationally expensive to scale to larger models. APD is also highly sensitive to its top-k hyperparameter, which specifies the expected number of active components per input and must be chosen in advance. Lastly, APD relies on the magnitude of the gradient as a proxy for the causal importance of each component. These gradients have been shown to be poor approximations of the true importance. (cite the papers here).

3.2 Stochastic parameter decomposition

Stochastic Parameter Decomposition (SPD) was introduced as a solution to the limitations of APD. SPD decomposes each weight matrix into a sum of rank-one matrices, called subcomponents:

$$W^l \approx \sum_{c=1}^{C} \vec{u}_c^l \vec{v}_c^{lT}, \quad \text{where} \quad \vec{u} \in \mathbb{R}^{d_{out}}, \vec{v} \in \mathbb{R}^{d_{in}}$$

The number of subcomponents C is a hyperparameter that may be chosen to be larger than the rank of W^l enabling the method to learn features in superposition [BBS25].

These subcomponents are then trained to be faithful to the original weights, to have as few active subcomponents as possible for any given input, and lastly to reconstruct the original network's output.

3.2.1 Faithfulness

3.2.2 Storchastic reconstruction

3.2.3 Minimality

3.2.4 Limitations

These components are later clustered manually.... -¿ GNaN solves Components are rank 1, many features are multidimensional, dependent components are trained independently

3.3 Some clever name for my additions

This works primary purpose is to address two methological limtations of SPD, namely the rank-one constraint on subcomponents and the independence of the importance functions across subcomponents. The necessity of these modifications will be demonstrated in the experiments, where dependent and multi-dimensional features will be shown to be common in even simple models, and the original SPD method will be shown to fail to recover them (i hope). Additionally, this work also provides a first step towards a general clustering mechanism that operates during training rather than a post-hoc step, by masking features of each subcomponent independently and then allowing related features to be clustered together (again i hope).

To achieve this, three modifications are introduced: rank-k subcomponents that can natively capture multi-dimensional mechanisms, per-dimension ablation that implicitly learns the rank of each mechanism, and an additive graph neural network based importance function that conditions each importance on the activation of neighbouring subcomponents.

3.3.1 Rank-k subcomponents

In SPD, each subcomponent is a rank-one outer product of two vectors. A mechanism of rank r must therefore be represented as r separate subcomponents, with no training signal to keep them coherent. To generalize this, each pair of vectors is replaced with a pair of matrices:

$$W^l \approx \sum_{c=1}^{C} U_c^l \, V_c^{l\top}, \quad \text{where} \quad U_c^l \in \mathbb{R}^{d_{\text{out}} \times k}, \; V_c^l \in \mathbb{R}^{d_{\text{in}} \times k}$$

Each subcomponent $U_c^l V_c^{l\top} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is a matrix of rank at most k. Setting $k = 1$ recovers the original SPD formulation.

As a result of this change, the inner activation used in the importance function becomes a k-dimensional vector instead of a single scalar:

$$\mathbf{h}_c^l(x) = V_c^{l\top} \, \vec{a}^{\,l}(x) \in \mathbb{R}^k$$

The extension from SPD follows naturally: the importance function is no longer derived from a single inner activation, but from k inner activations, which are then processed jointly to determine whether the mechanism should remain active.

3.3.2 GNaN-based importance functions

In SPD, each importance function γ_c^l is an independent MLP that receives only its own inner activation. Even with the richer k-dimensional inner activations introduced in Section 3.3.1, this independence prevents subcomponents from using cross-component information to disambiguate signal from interference. We replace the independent MLPs with a Graph Neural Additive Network (GNaN) that operates over the set of subcomponents.

The C subcomponents are treated as nodes in a graph, with node features given by their inner activation vectors $\mathbf{h}_c^l(x) \in \mathbb{R}^k$. The GNaN updates each node's representation through an additive aggregation. For node i and feature dimension k:

$$[\mathbf{h}_i]_k = \sum_{j=1}^{N} \frac{1}{\#\text{dist}_i(j,i)} \cdot \rho\left(\frac{1}{1+\text{dist}(j,i)}\right) \cdot f_k([\mathbf{x}_j]_k)$$

where f_k is a learned per-feature function applied to the k-th inner activation of neighbour j; $\text{dist}(j,i)$ is a distance function computed from the layer distance between subcomponents j and i and the cosine similarity of their parameter vectors; ρ is a learned function that maps the distance kernel to a scalar weight; and $\#\text{dist}_i(j,i)$ normalises by the number of nodes at the same distance from i as j. The update is residual:

$$\mathbf{h}_i \leftarrow \mathbf{h}_i + \text{GNaN}(\mathbf{h}_i, \{\mathbf{h}_j\}_{j \neq i})$$

The additive structure is a deliberate choice. Each feature dimension k is processed independently by f_k, and each neighbour's contribution is weighted independently by the distance function. This preserves interpretability of the importance computation: the contribution of each neighbour to each feature dimension can be inspected individually.

After the GNaN update, the per-dimension importance values are computed from the updated representation and passed through a leaky hard sigmoid, following SPD [BBS25]:

$$\mathbf{g}_c^l(x) = \sigma_H\big(\gamma_c\big(\mathbf{h}_c^l(x)\big)\big) \in [0,1]^k$$

This architecture addresses both limitations of SPD simultaneously. The rank-k subcomponents eliminate the need for post-hoc clustering by capturing multi-dimensional mechanisms as single units. The per-dimension ablation learns the effective rank of each mechanism automatically. And the GNaN enables importance decisions to be conditioned on the full neighbourhood, allowing subcomponents to reject interference from other active mechanisms and to coordinate their importance values when they participate in shared computations.

3.3.3 Per-dimension ablation and implicit rank selection

A natural concern with rank-k subcomponents is the choice of k: if k is smaller than the true rank of a mechanism, the subcomponent cannot capture it fully; if k is larger, the subcomponent has excess capacity that could absorb unrelated computations.

We resolve this by allowing the training objective to determine the effective rank of each subcomponent automatically. Rather than applying a single scalar mask to the entire rank-k block, we mask each of the k dimensions independently. The importance function outputs a k-dimensional vector:

$$\mathbf{g}_c^l(x) \in [0,1]^k$$

and each dimension receives its own stochastic mask:

$$[m_c^l(x,r)]_j = [g_c^l(x)]_j + \big(1 - [g_c^l(x)]_j\big)\, r_{c,j}^l, \qquad r_{c,j}^l \sim U(0,1)$$

for $j = 1, \ldots, k$. The masked weight matrix becomes:

$$W'^l(x,r) = \sum_{c=1}^{C} U_c^l \,\mathrm{diag}\big(\mathbf{m}_c^l(x,r)\big)\, V_c^{l\top}$$

The importance minimality loss penalises all k dimensions of all subcomponents toward zero:

$$\mathcal{L}_{\text{import}} = \sum_{l=1}^{L}\sum_{c=1}^{C}\sum_{j=1}^{k} \big|[g_c^l(x)]_j\big|^p$$

This follows the same logic by which SPD selects the number of active subcomponents: the minimality loss drives unused components toward zero, and only those that are genuinely required to reconstruct the target model's output survive. Here, the same pressure is applied one level deeper — within each subcomponent. If a mechanism is truly rank-r with $r < k$, the remaining $k - r$ dimensions will be driven to zero importance, as they can be ablated without affecting the output.

This makes k a soft upper bound rather than a sensitive hyperparameter. In practice, k can be set conservatively large and the effective rank of each subcomponent is learned from the data.

3.3.4 Training objective

The full loss function retains the same structure as SPD:

$$\mathcal{L} = \mathcal{L}_{\text{faith}} + \beta_1 \mathcal{L}_{\text{stoch}} + \beta_2 \mathcal{L}_{\text{stoch-layer}} + \beta_3 \mathcal{L}_{\text{import}}$$

The faithfulness loss trains the sum of rank-k subcomponents to approximate the original weight matrices:

$$\mathcal{L}_{\text{faith}} = \frac{1}{N}\sum_{l=1}^{L} \left\| W^l - \sum_{c=1}^{C} U_c^l\, V_c^{l\top} \right\|_F^2$$

The stochastic reconstruction loss trains the masked model to replicate the target model's output:

$$\mathcal{L}_{\text{stoch}} = \frac{1}{S}\sum_{s=1}^{S} D\Big(f(x \mid W'(x, r^{(s)})),\ f(x \mid W)\Big)$$

where D is an appropriate divergence measure and S is the number of mask samples per training step. The layerwise variant $\mathcal{L}_{\text{stoch-layer}}$ applies stochastic masking to one layer at a time while keeping all other layers at their original weights.

The importance minimality loss operates over all k dimensions of all subcomponents as defined in Section 3.3.2. The hyperparameters are $\beta_1, \beta_2, \beta_3$ (loss coefficients), p (the norm used in $\mathcal{L}_{\text{import}}$), S (mask samples per step), k (the maximum rank of each subcomponent), and C (the number of subcomponents per layer).

Together, the three modifications form a complementary system. Rank-k subcomponents provide the *capacity* to represent multi-dimensional mechanisms — a single subcomponent can host a feature that occupies an r-dimensional subspace. Per-dimension ablation provides the *pressure* to discard unused capacity — dimensions that do not contribute to the network's output are suppressed, preventing a subcomponent from absorbing unrelated computations. The GNaN provides the *routing*: by conditioning each importance decision on the activation state of neighbouring subcomponents, it can learn to consolidate a multi-dimensional feature into a single subcomponent whose capacity is sufficient to host it, rather than fragmenting it across several. In this sense, the GNaN serves as an implicit clustering mechanism that operates during training rather than as a post-hoc step.

The guiding principle behind these methods is one of minimum description length — a good decomposition should identify the fewest, simplest components needed to faithfully explain what the network is doing on any given input.

3.4 Dead salmon thing

4 Experiments

4.1 Toy model of superposition

4.2 Rank-r Toy Model of Superposition

To evaluate whether [METHOD_NAME] can recover multi-dimensional mechanisms and implicitly learn their rank, we introduce a generalisation of the Toy Model of Superposition (TMS) [**ToyModelsSuperposition**] in which features are r-dimensional vectors rather than scalars. We conduct two experiments: one in which all features share the same rank, and one in which features of different ranks coexist.

4.2.1 Setup

The standard TMS reconstructs sparse scalar inputs through a bottleneck:

$$\hat{\mathbf{x}} = \mathrm{ReLU}(W^\top W\mathbf{x} + \mathbf{b}), \quad W \in \mathbb{R}^{m\times n}, \quad m < n$$

Each feature $x_i \in \mathbb{R}$ is a scalar, and each column $\vec{w}_i$ of W is the rank-one mechanism that encodes it. We generalise this by replacing each scalar feature with a vector $\mathbf{x}^{(i)} \in \mathbb{R}^{r_i}$, where r_i is the rank of feature i. The full input is the concatenation $\mathbf{x} = (\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)})^\top \in \mathbb{R}^d$, where $d = \sum_{i=1}^n r_i$. The model becomes:

$$\hat{\mathbf{x}} = \mathrm{ReLU}(W^\top W\mathbf{x} + \mathbf{b}), \quad W \in \mathbb{R}^{m\times d}, \quad m < d$$

The columns of W are grouped into n blocks:

$$W = \left[\underbrace{W_1}_{r_1 \text{ cols}} \mid \underbrace{W_2}_{r_2 \text{ cols}} \mid \cdots \mid \underbrace{W_n}_{r_n \text{ cols}} \right]$$

where $W_i \in \mathbb{R}^{m\times r_i}$ is the embedding for feature i. The ground truth mechanism for feature i is the rank-r_i matrix $W_iW_i^\top$, which is active if and only if $\mathbf{x}^{(i)} \neq \mathbf{0}$.

4.2.2 Data distribution

Each feature is independently active with probability $s \ll 1$. When active, it is sampled uniformly from the unit sphere S^{r_i-1} to ensure that all r_i dimensions are necessary for reconstruction:

$$\mathbf{x}^{(i)} \sim \begin{cases} \mathrm{Unif}(S^{r_i-1}) & \text{with probability } s \\ \mathbf{0} \in \mathbb{R}^{r_i} & \text{with probability } 1-s \end{cases}$$

The target model is trained with an importance-weighted reconstruction loss:

$$\mathcal{L}_{\text{target}} = \sum_{i=1}^{n} \lambda_i \left\|\hat{\mathbf{x}}^{(i)} - \mathbf{x}^{(i)}\right\|^2$$

where λ_i are importance weights and $\hat{\mathbf{x}}^{(i)}$ is the reconstruction of the i-th block.

4.2.3 Experiment 1: Uniform rank

In the first experiment, all features share the same rank $r_i = r$ for all i. We fix $n = 40$ features, hidden dimension $m = 15$, sparsity $s = 0.05$, and set $k = 8$ for the decomposition. We train target models for $r \in \{1, 2, 4, 8\}$ and decompose each with three methods:

- SPD (rank-one subcomponents, independent gates),

METHOD_NAME without GNaN (rank-k subcomponents with per-dimension ablation, independent gates),
METHOD_NAME (rank-k subcomponents with per-dimension ablation and GNaN).

For each decomposition, we measure:

- **MMCS**: Mean max cosine similarity between recovered and ground truth mechanisms, adapted to rank-r subspaces by comparing the principal angles between the column spans of W_i and the learned subcomponents.
- **Effective rank**: The number of dimensions per subcomponent whose mean importance $\mathbb{E}_x[|[g_c^l(x)]_j|]$ exceeds a threshold $\tau = 0.5$.
- **Reconstruction loss**: $D(f(x \mid W'), f(x \mid W))$ on held-out data.

The prediction is that at $r = 1$, all three methods perform comparably, since all mechanisms are rank-one and the extensions provide no advantage. As r increases, SPD should degrade because its independent scalar gates must coordinate across r fragments, while [METHOD_NAME] should maintain performance by capturing each mechanism as a single rank-r block. The effective rank per subcomponent should match r, with the remaining $k - r$ dimensions driven to zero by the importance minimality loss.

4.2.4 Experiment 2: Mixed rank

In the second experiment, features have heterogeneous ranks. We set $n = 40$ features, of which 20 are rank-one ($r_i = 1$) and 20 are rank-two ($r_i = 2$). The hidden dimension is $m = 15$, sparsity $s = 0.05$, and $k = 4$ for the decomposition. All other settings are identical to Experiment 1.

This experiment tests whether the implicit rank selection operates correctly when the true ranks vary across features within the same model. The prediction is that [METHOD_NAME] learns subcomponents whose effective rank matches the ground truth rank of the feature they capture: subcomponents corresponding to rank-one features should have one active dimension and $k - 1$ suppressed dimensions, while those corresponding to rank-two features should have two active dimensions and $k - 2$ suppressed dimensions.

We visualise this by plotting the per-dimension importance values $\mathbb{E}_x[|[g_c^l(x)]_j|]$ for each subcomponent, sorted by magnitude. For a successful decomposition, this should produce a bimodal pattern: a cluster of subcomponents with one active dimension and a cluster with two, with a clear gap between the active and suppressed dimensions within each subcomponent.

4.3 Toy model with relu

4.4 Smallest possible LM

5 Discussion

6 References

7 Other things to consider?

Minimal descriptive length?

7.1 That some features are multidimensional

Think back to SAE, that learn a dictionary of features, then find hte linear combination of those features, however that this fails for cyclical features, as its cheaper to learn a single feature, than learn a direction and a periodic function. Now for your experiment, the dream scenario is that we allowe a multi dimensional component, and that this can learn

the cyclical feature, i.e a weekly cycle or modular arithmatic either do this or extend TMS to be multi-dimensional, show if it works better with ur gnn or not, likewise we need to show that through individual masking of features, we can teach a component to kill one of its features, if it is better explained by a rank 1 component than rank 2.

[BBS25] Lucius Bushnaq, Dan Braun, and Lee Sharkey. *Stochastic Parameter Decomposition*. 2025. arXiv: 2506.20790 [cs.LG]. URL: https://arxiv.org/abs/2506.20790.

[Bra+25] Dan Braun et al. *Interpretability in Parameter Space: Minimizing Mechanistic Description Length with Attribution-based Parameter Decomposition*. 2025. arXiv: 2501.14926 [cs.LG]. URL: https://arxiv.org/abs/2501.14926.

[Cha+25] David Chanin et al. *A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders*. 2025. arXiv: 2409.14507 [cs.CL]. URL: https://arxiv.org/abs/2409.14507.

[Cun+23] Hoagy Cunningham et al. *Sparse Autoencoders Find Highly Interpretable Features in Language Models*. 2023. arXiv: 2309.08600 [cs.LG]. URL: https://arxiv.org/abs/2309.08600.

[Lea+25] Patrick Leask et al. *Sparse Autoencoders Do Not Find Canonical Units of Analysis*. 2025. arXiv: 2502.04878 [cs.LG]. URL: https://arxiv.org/abs/2502.04878.